PREDICTING LEVELS OF DAMAGES TO BUILDINGS CAUSED BY EARTHQUAKE

A PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted by

NAME		REG. No.
ARULSELVAM C	–	211011005
GOWTHAM R	–	211011014
PRANAV VARSHAN A T	–	211011035
RAJADURAI P	–	211011038

Project Guide

Dr. R. RAGUPATHY

Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ANNAMALAI UNIVERSITY

ANNAMALAINAGAR – 608 002

April 2025

ANNAMALAI UNIVERSITY

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Project report titled **"PREDICTING LEVELS OF DAMAGES TO BUILDINGS CAUSED BY EARTHQUAKE"** is the bonafide record of the work done by

NAME		REG. No.
ARULSELVAM C	**–**	**211011005**
GOWTHAM R	**–**	**211011014**
PRANAV VARSHAN A T	**–**	**211011035**
RAJADURAI P	**–**	**211011038**

in partial fulfillment of the requirements for the Degree of Bachelor of Engineering in Computer Science and Engineering (Data Science) during the period 2024 - 2025.

Dr. R. RAGUPATHY
Professor
Dept. of Computer Science & Engg.
Project Guide

Dr. R. BHAVANI
Professor & Head
Dept. of Computer Science & Engg.

Internal Examiner

External Examiner

Place: Annamalai Nagar
Date:

ACKNOWLEDGEMENT

We wish to express our sincere thanks and deep sense of gratitude to **Dr. R. BHAVANI, M.E., Ph.D., Professor & Head**, Department of Computer Science and Engineering, Faculty of Engineering and Technology, Annamalai University for giving us the opportunity to undertake this project.

We would like to convey our heartiest thanks to our project guide **Dr. R. RAGUPATHY, M.E., Ph.D., Professor**, Department of Computer Science and Engineering, for all him help and support. He with him extreme patience has guided us in situation of need for which we are extremely grateful.

We would like to thank our project coordinator **Dr. L.R. SUDHA, M.E., Ph.D., Associate Professor** and review committee member **Dr. P. SUDHAKAR, M.E., M.B.A., Ph.D., Professor,** Department of Computer Science and Engineering for their great support and encouragement during the course of our project.

We would like to thank all our friends for their support in time of need, encouragement and were always ready to help us without asking for it.

We wish to thank all the technical staff members, incharge of our department laboratories that fulfilled all our project needs and offered us timely help.

Above all, we are indebted to our beloved parents, whose blessings and best wishes have come a long way in making this final year project work a grand success.

ARULSELVAM C

GOWTHAM R

PRANAV VARSHAN A T

RAJADURAI P

ABSTRACT

The 2015 Nepal earthquake, a catastrophic 7.8 magnitude event, caused severe infrastructure damage and a tragic loss of life. This disaster underscored the urgent need for improved disaster management and preparedness. Predicting building damage levels post-earthquake is critical for effective response and resource allocation during such crises. This project utilizes deep learning models to predict damage levels to buildings using features such as structural details, building materials, and geographical information. By analyzing data from the 2015 Nepal earthquake, the project aims to understand patterns in building vulnerability. The insights generated will help authorities prioritize rescue operations and allocate resources efficiently. Additionally, the findings can guide urban planning to minimize risks in earthquake-prone areas. This work enhances disaster resilience and preparedness, ensuring safer communities and improving disaster response strategies for future seismic events.

திட்டச்சுருக்கம்

2015 நேபாள நிலநடுக்கம், 7.8 ரிக்டர் அளவிலான பேரழிவு நிகழ்வு, கடுமையான உள்கட்டமைப்பு சேதத்தையும் சோகமான உயிர் இழப்பையும் ஏற்படுத்தியது. பேரிடர் மேலாண்மை மற்றும் தயார்நிலையில் மேம்படுத்தப்பட வேண்டியதன் அவசியத்தை இந்தப் பேரிடர் அடிக்கோடிட்டுக் காட்டுகிறது. நிலநடுக்கத்திற்குப் பிந்தைய கட்டிட சேத அளவுகளை கணிப்பது இதுபோன்ற நெருக்கடிகளின் போது பயனுள்ள பதில் மற்றும் வள ஒதுக்கீட்டிற்கு முக்கியமானது. கட்டமைப்பு விவரங்கள், கட்டுமானப் பொருட்கள் மற்றும் புவியியல் தகவல்கள் போன்ற அம்சங்களைப் பயன்படுத்தி கட்டிடங்களுக்கு ஏற்படும் சேத நிலைகளை கணிக்க இந்த திட்டம் ஆழ்ந்த கற்றல் மாதிரிகளைப் பயன்படுத்துகிறது. 2015 நேபாள பூகம்பத்தின் தரவை பகுப்பாய்வு செய்வதன் மூலம், இந்த திட்டம் கட்டிட பாதிப்பில் உள்ள வடிவங்களைப் புரிந்துகொள்வதை நோக்கமாகக் கொண்டுள்ளது. உருவாக்கப்பட்ட நுண்ணறிவுகள் மீட்பு நடவடிக்கைகளுக்கு முன்னுரிமை அளிக்கவும், வளங்களை திறமையாக ஒதுக்கவும் அதிகாரிகளுக்கு உதவும். கூடுதலாக, கண்டுபிடிப்புகள் பூகம்பத்தால் பாதிக்கப்படக்கூடிய பகுதிகளில் அபாயங்களைக் குறைக்க நகர்ப்புற திட்டமிடலுக்கு வழிகாட்டும். இந்த வேலை பேரழிவு பின்னடைவு மற்றும் தயார்நிலையை மேம்படுத்துகிறது, பாதுகாப்பான சமூகங்களை உறுதி செய்கிறது மற்றும் எதிர்கால நில அதிர்வு நிகழ்வுகளுக்கான பேரழிவு பதிலளிப்பு உத்திகளை மேம்படுத்துகிறது.

TABLE OF CONTENTS

CHAPTER NO.	CONTENT		PAGE NO.
	Abstract		i
	திட்டச்சுருக்கம்		ii
	List of Figures		iii
	List of Tables		iv
	List of Abbreviations		v
1	**INTRODUCTION**		
	1.1	Problem Statement	2
	1.2	Objective(s) and Organization of the Report	3
2	**LITERATURE SURVEY**		5
3	**PROPOSED METHODOLOGY**		
	3.1	System Architecture and Block Diagram	8
	3.2	System Requirements	11
	3.3	Deep Learning Modules	11
	3.4	Summary and Comparative Overview	14
4	**IMPLEMENTATION**		
	4.1	Dataset Description	16
	4.2	Data Preprocessing	17
	4.3	Model Implementation	18
	4.4	Hyperparameter Tuning	29

	4.5	Evaluation Metrics	30
	4.6	Experiments and Results	34
	4.7	Interpretation of Results	37
5	**CONCLUSION**		41
	REFERENCES		43

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	System Architecture and Block Diagram	8
4.1	The Convolutional Nueral Network Architecture	18
4.2	The Bidirectional Long Short-Term Memory Architecture	20
4.3	The Graph-Based Neural Network Architecture	21
4.4	The Tabular Neural Network Architecture	23
4.5	The Tabular Prior-Data Fitted Network Architecture	26
4.6	The Neural Oblivious Decision Ensembles Architecture	27
4.7	CNN Model Accuracy and Loss Trends Over Epochs	34
4.8	Confusion Matrix for CNN Model	34
4.9	GBNN Model Accuracy and Loss Trends Over Epochs	35
4.10	Confusion Matrix for GBNN Model	35
4.11	BLSTM Model Accuracy and Loss Trends Over Epochs	35
4.12	Confusion Matrix for BLSTM Model	35
4.13	TabNet Model Accuracy and Loss Trends Over Epochs	35
4.14	Confusion Matrix for TabNet Model	35
4.15	TabPFN Model Accuracy and Loss Trends Over Epochs	36
4.16	Confusion Matrix for TabPFN Model	36
4.17	NODE Model Accuracy and Loss Trends Over Epochs	36
4.18	Confusion Matrix for NODE Model	36
4.19	Model Performance Comparison	37

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
3.1	Summary of Deep Learning and Hybrid Models with Applications in Earthquake Damage Prediction	15
4.1	Summarizes the Results for All Models	37

LIST OF ABBREVIATIONS

S. NO	ABBREVIATION	EXPANDED FORM
1	AI	Artificial Intelligence
2	Agg	Aggregate
3	BLSTM	Bidirectional Long Short-Term Memory Networks
4	BN	Batch Normalization
5	CNN	Convolutional Neural Networks
6	DL	Deep Learning
7	FN	False Negative
8	FP	False Positive
9	GAT	Graph Attention Network
10	GBNN	Graph Based Neural Networks
11	GCN	Graph Convolution Network
12	GPS	Global Positioning System
13	LSTM	Long Short-Term Memory
14	ML	Machine Learning
15	MLP	Multilayer Perceptron
16	NODE	Neural Oblivious Decision Ensembles
17	ReLU	Rectified Linear Unit
18	SMOTE	Synthetic Minority Over-Sampling Techniques
19	TabNet	Tabular Neural Network
20	TabPFN	Tabular Prior-Data Fitted Network

21	TN	True Negative
22	TP	True Positive

CHAPTER 1
INTRODUCTION

Earthquakes are among the most catastrophic natural disasters experienced across the globe, known for their sudden and destructive nature. These seismic events not only pose a significant threat to human lives but also lead to widespread damage to infrastructure, public utilities, and economic systems. Countries located in the seismic belts such as Nepal, Japan, Indonesia, and parts of India are especially vulnerable to these geological phenomena.

One of the key issues during and after an earthquake is the ability to assess the extent of damage to buildings in a timely and accurate manner. Traditional damage assessment processes often involve on-site inspection by structural engineers and survey teams. While this manual method offers the benefit of professional judgment, it is also time-consuming, labor-intensive, and risky during the post-disaster chaos when time is of the essence. Moreover, delays in damage assessment may hinder prompt rescue and recovery operations, potentially resulting in avoidable casualties and further infrastructure loss.

In recent years, technological advancements have offered promising solutions in the form of Artificial Intelligence (AI) and Machine Learning (ML). These fields have revolutionized how data can be interpreted and predictions made, offering efficiency and scalability not possible through manual approaches. Deep Learning (DL), a subset of ML, excels in learning complex, non-linear patterns from vast amounts of data and has demonstrated superior performance in various domains including image recognition, natural language processing, and medical diagnostics.

The focus of this project lies in leveraging deep learning and transformer-based architectures to predict the level of structural damage to buildings caused by earthquakes. Specifically, we make use of the 2015 Nepal Gorkha Earthquake dataset, which contains extensive information on over 25,000 buildings affected by the earthquake. This dataset

includes critical features such as the number of floors, building age, construction material, foundation type, geographic location, and more. Such detailed information allows for a robust exploration of patterns and correlations relevant to earthquake damage.

Several advanced models are explored and implemented in this project, including Convolutional Neural Networks (CNN), Bidirectional Long Short-Term Memory networks (BLSTM), Graph-Based Neural Networks (GBNN), and transformer-based tabular data models like Tabular Neural Network (TabNet), Tabular Prior-Data Fitted Network (TabPFN), and Neural Oblivious Decision Ensembles (NODE). Each model brings unique strengths to the table, enhancing the prediction capabilities through various architectural and learning innovations. The ultimate goal is to develop a system that aids emergency response teams and urban planners by providing rapid and accurate assessments, which can be critical in saving lives and optimizing the deployment of limited resources.

1.1 PROBLEM STATEMENT

Predicting the level of structural damage to buildings during earthquakes presents several critical challenges. Foremost among these is the classification of damage levels into clearly defined categories such as low, medium, and high. In the chaotic aftermath of an earthquake, quick and reliable information on building damage can dramatically improve response time and resource allocation. However, manual inspection is not only inefficient but often impossible due to accessibility issues and the scale of devastation.

Existing ML approaches offer some improvements over manual assessments, yet they struggle with specific technical limitations. Many traditional ML algorithms are not well-equipped to manage high-dimensional and imbalanced datasets—two common characteristics of real-world disaster datasets. Additionally, these models often lack the flexibility to adapt to the complex relationships and contextual interactions between structural features and geographical conditions.

To overcome these limitations, this project proposes the use of advanced DL models that can better capture the non-linear dependencies and hierarchical structures within the data. DL models can generalize well on large datasets and have demonstrated impressive results

in predictive tasks involving classification and regression. Moreover, the integration of transformer-based models tailored for tabular data enables improved attention to relevant features and higher accuracy across class distributions.

The specific problem we address in this project is the development of a reliable and scalable AI-driven system that classifies buildings into one of three damage categories based on their structural and geographical attributes. This system must handle a moderately imbalanced dataset and maintain high accuracy, particularly in detecting severely damaged buildings which are often underrepresented in the data.

1.2 OBJECTIVE(S) AND ORGANIZATION OF THE REPORT

Objectives:

The primary aim of this project is to develop a machine learning system that can predict the extent of building damage caused by earthquakes using deep learning and transformer-based models. The specific objectives include:

- To perform data preprocessing and feature engineering on the 2015 Nepal Earthquake dataset to prepare it for model training and evaluation.
- To implement a range of advanced deep learning architectures including CNN, BLSTM, GBNN, TabNet, TabPFN, and NODE.
- To evaluate each model based on standard performance metrics such as accuracy, precision, recall, and F1-score, with a goal of achieving at least 74.5% predictive accuracy.
- To compare the effectiveness of traditional deep learning models with transformer-based approaches in the context of earthquake damage prediction.
- To provide insights into model interpretability and decision support that can assist government agencies, emergency responders, and urban developers in disaster response planning.

Organization of the Report:

This project report is organized into five main chapters:

- Chapter 1 – Introduction: Describes the background, problem statement, objectives, and scope of the project.
- Chapter 2 – Literature Survey: Reviews the existing works related to earthquake damage prediction and evaluates the strengths and weaknesses of different ML and DL models.
- Chapter 3 – Methodology: Provides an in-depth explanation of the proposed approach, including the system architecture, model selection, and training strategy.
- Chapter 4 – Implementation: Details the dataset characteristics, data preprocessing steps, model implementation, and evaluation methodology. Result Analysis: Presents experimental results, performance comparisons, and discusses the implications of findings.
- Chapter 5 – Conclusion and Future Work: Summarizes the outcomes of the project and outlines possible future directions for improvement and extension.

CHAPTER 2
LITERATURE SURVEY

Earthquake damage prediction has seen a surge of interest with the increasing availability of post-disaster data and the growing capabilities of AI and remote sensing technologies. The evolution from manual inspection toward automated damage assessment using machine learning and ensemble models has led to promising results in both urban and regional-scale seismic impact prediction. The following literature highlights the key advancements relevant to this study:

In [1], Breiman L. proposed a groundbreaking ensemble technique known as Bagging Predictors. This approach improves prediction accuracy by combining multiple versions of a predictor trained on bootstrapped subsets of the data. The method showed significant error reduction for unstable models such as decision trees, especially when used for classification and regression tasks.

In [2], Bai Y., Adriano B., Mas E., and Koshimura S. evaluated the use of dual-polarization synthetic aperture radar (SAR) imagery for building damage assessment after the 2015 Nepal Gorkha earthquake. They employed texture-based features and support vector machines, achieving 80.5% classification accuracy without requiring pre-event images.

Ghimire S., Gueguen P., Giffard-Roisin S., and Schorlemmer D. [3] tested machine learning models for seismic damage prediction using the extensive post-2015 Nepal earthquake building database. The study found that random forest models outperformed others in predicting building damage grades based on basic features like number of stories, height, age, and plinth area.

In [4], Breiman L. further developed the concept of ensemble models through Random Forests, an extension of decision trees that introduces random feature selection at each split. This model demonstrated improved robustness and lower generalization error across multiple datasets, becoming a key tool in damage prediction modeling.

In [5], Dong L. and Shan J. provided a comprehensive review of remote sensing techniques for detecting earthquake-induced building damage. The study categorizes methods into multi-temporal and mono-temporal techniques, analyzing their performance using optical, SAR, and LiDAR datasets. This work is pivotal in understanding the data-driven approaches for disaster response.

In [6], Rao A., Jung J., Silva V., Molinario G., and Yun S-H. proposed a supervised machine learning framework that integrates high-resolution SAR data, ShakeMaps, and building footprint information to classify post-earthquake damage. Using both binary and multi-class classifications, they achieved notable accuracy improvements, setting a benchmark for rapid disaster mapping.

In [7], Kahandawa K.A.R.V.D., Domingo N.D., Park K.S., and Uma S.R. conducted a comprehensive literature review on earthquake damage estimation systems. Their study highlights various data-driven methodologies using GIS, remote sensing, and machine learning, emphasizing the challenges of data quality, integration, and interoperability.

Dong L. and Shan J. [8] presented a detailed overview of remote sensing-based building damage detection techniques. They focused on the advantages of optical imagery, SAR, and LiDAR data, and proposed a fusion model for increased detection accuracy in urban settings.

In [9], Das P.K., Dutta S.C., and Sengupta P. analyzed rapid visual screening (RVS) methods deployed after major Indian earthquakes. The study identified key gaps in uniformity, risk prioritization, and recommended integrating these systems with AI for scalability.

Masi A., Chiauzzi L., Santarsiero G., Liuzzi M., and Tramutoli V. [10] proposed a remote sensing and field survey-based hybrid model to recognize seismic damage. Using case studies from the 2016 Central Italy earthquake, they validated the reliability of multi-source integration.

In [11], Wang X. and Li P. devised a damage detection approach using spectral, height, and corner information extracted from high-resolution satellite imagery and airborne LiDAR. Their system enabled fine-grained analysis for rapid urban risk assessment.

Khodaverdi N., Rastiveis H., and Jouybari A. [12] proposed a data fusion strategy combining post-earthquake satellite imagery and LiDAR to detect building collapses. Their study demonstrated improved detection rates, particularly in dense cityscapes.

Saganeiti L., Amato F., Nolè G., Vona M., and Murgante B. [13] presented a method for early ground displacement estimation using SAR and LiDAR data. Applied during the Amatrice earthquake, their framework provided a reliable correlation with observed structural damage.

Syifa M., Kadavi P.R., and Lee C.-W. [14] proposed an AI-based system for post-earthquake damage mapping in Palu, Indonesia. By leveraging multi-temporal satellite images, their model effectively classified heavily damaged regions using deep learning.

In [15], Ma H., Liu Y., Ren Y., and Yu J. developed an enhanced YOLOv3 algorithm to identify collapsed structures from remote sensing images. Their modification significantly improved object detection performance for earthquake-impacted regions.

Lastly, Turker M. and Sumer E. [16] explored post-earthquake building-based damage detection using watershed segmentation techniques. Their approach successfully distinguished between damaged and undamaged buildings using aerial imagery.

CHAPTER 3
PROPOSED METHODOLOGY

This project aims to predict the structural damage levels to buildings caused by earthquakes using a series of deep learning and transformer-based models. Using the 2015 Nepal Earthquake dataset as a foundation, the pipeline includes modules for data preprocessing, feature engineering, model development, evaluation, and result interpretation. Models are trained to classify buildings into low, medium, or high damage categories based on their structural and geospatial features.

3.1 SYSTEM ARCHITECTURE AND BLOCK DIAGRAM

The system architecture is designed as a modular, end-to-end pipeline consisting of six interconnected components that collectively enable efficient preprocessing, model training, evaluation, and prediction for earthquake-induced building damage classification.

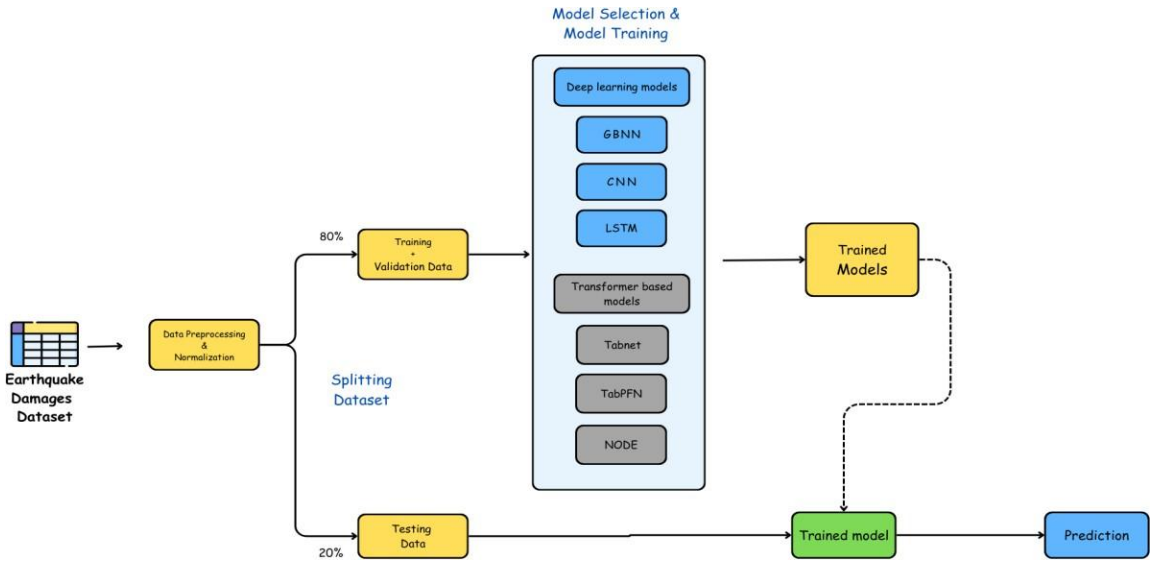

Fig 3.1: System Architecture and Block Diagram

3.1.1 DATA PREPROCESSING MODULE

The preprocessing steps used in this project included multiple critical operations to ensure data readiness and optimal model performance:

- **Missing Value Handling**: Basic imputation methods were applied wherever necessary to address incomplete data entries. Missing values were carefully handled using default replacements based on feature types, ensuring the overall distribution of the data remained intact without introducing bias.
- **Encoding Categorical Features**: Label Encoding was applied for ordinal features where the order of categories matters, while One-Hot Encoding was used for nominal features like roof_type, foundation_type, and land_surface_condition. This transformation allowed machine learning models to interpret categorical data correctly without misinterpreting any inherent order.
- **Feature Scaling**: Numerical features were scaled using Min-Max normalization, which rescaled each feature to a range between 0 and 1. This scaling was crucial to ensure that models like CNNs and LSTMs, which are sensitive to feature magnitude, could learn effectively without dominance by features of larger numerical ranges.

3.1.2 DATASET SPLITTING MODULE

This module divides the processed dataset into training and testing sets in an 80:20 ratio while carefully preserving the original class distribution to maintain model reliability. Stratified sampling techniques are utilized to ensure that each damage grade class is proportionally represented in both subsets, which is crucial for fair model evaluation. Proper dataset splitting helps avoid issues like data leakage, where information from the testing set could inadvertently influence the training phase. It also ensures that the performance metrics computed during evaluation accurately reflect the model's ability to generalize to unseen data.

3.1.3 MODEL SELECTION AND TRAINING MODULE

This module is responsible for implementing, configuring, and training multiple machine learning and deep learning models independently. It covers the full training pipeline for a variety of deep learning models including CNN, BLSTM, GBNN, and transformer-based models such as TabNet, TabPFN, and NODE. Each model is carefully designed and compiled with appropriate loss functions and optimizers suited for multi-class classification tasks. During training, hyperparameters like learning rate, batch size, and the number of epochs are tuned to optimize performance. The module also ensures validation during training to monitor overfitting and underfitting issues. Extensive experimentation across these architectures enables the development of robust and high-accuracy models for earthquake building damage classification.

3.1.4 MODEL EVALUATION MODULE

This module is dedicated to assessing the performance and reliability of each trained model. It uses critical evaluation metrics including accuracy, precision, recall, and F1-score to measure the classification effectiveness across different damage grades. Accuracy provides an overall correctness measure, precision evaluates the exactness of positive predictions, recall measures the model's ability to capture all positive instances, and the F1-score balances precision and recall. Additionally, confusion matrices are constructed and analyzed to visualize how well the models distinguish between different classes. Proper evaluation helps in identifying specific strengths and weaknesses of each model, guiding the selection of the most robust and generalizable model for final deployment.

3.1.5 PREDICTION MODULE

This module uses the best-performing trained model, selected after thorough evaluation, to predict the damage grades of buildings based on unseen input data. Once a model is deployed, it processes new feature sets representing structural and geospatial characteristics and produces a classification output indicating the expected level of damage—categorized as low, medium, or high. The prediction module is designed to operate efficiently, providing timely and accurate results that can support disaster management authorities in prioritizing recovery efforts. Moreover, the prediction results

can be stored, visualized, and integrated into broader decision support systems for further analysis or real-time alerting mechanisms in future implementations.

3.2 SYSTEM REQUIREMENTS

3.2.1 HARDWARE REQUIREMENTS:

- Processor: Intel i5 or above / Ryzen 5 or above
- RAM: 8 GB minimum (16 GB recommended)
- GPU: NVIDIA GTX 1650 or higher (for model training)
- Storage: 20 GB free space

3.2.2 SOFTWARE REQUIREMENTS:

- OS: Windows 10 / Ubuntu 20.04 or later
- Python 3.8+
- Google Colab / Google Kaggle
- Libraries: TensorFlow, PyTorch, Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn

3.3 DEEP LEARNING MODELS

3.3.1 CONVOLUTION NEURAL NETWORK

A deep learning architecture originally developed for processing and analyzing visual imagery, characterized by its ability to extract spatial hierarchies of features through convolutional layers. This project, CNNs are adapted for structured tabular data by reshaping the features into pseudo-image formats to enable local feature interaction learning. Multiple convolutional layers are stacked to progressively capture complex patterns and relationships among the input features. Pooling layers are incorporated to downsample feature maps, reduce computational complexity, and control overfitting. Finally, fully connected layers are employed to consolidate extracted features and perform multi-class classification, predicting the building damage grade. This adaptation effectively leverages the powerful feature extraction capabilities of CNNs, even in non-image data scenarios, significantly enhancing model performance.

3.3.2 BIDIRECTIONAL LONG SHORT-TERM MEMORY

BLSTM: An advanced and enhanced variant of the Long Short-Term Memory (LSTM) network that processes input sequences in both forward and backward directions simultaneously, enabling the model to capture contextual information from both the past and the future for every input feature. This dual flow of information allows the BLSTM to better understand complex patterns and intricate dependencies that may not be observable when processing sequential data in a single direction alone. Such bidirectional processing is highly advantageous in scenarios where the relationship between features is not strictly linear or where historical and future data points influence outcomes equally. In the context of earthquake building damage prediction, this is particularly useful for capturing relational dependencies between building attributes such as age, material, construction type, and location. BLSTM networks are highly effective when dealing with sequential, time-series, or ordered tabular data where temporal or logical ordering is critical to accurate modeling. They help significantly enhance the model's ability to detect, learn, and predict complex non-linear patterns with greater precision, improving the reliability of the final predictive outcomes.

3.3.3 GRAPH-BASED NEURAL NETWORK

GBNN: A deep learning model specifically designed to operate on graph-structured data, where each building is represented as a node and the relationships between buildings are modeled as edges based on either feature similarities or geographic proximity. This allows the model to naturally encode spatial relationships and interdependencies that are typically difficult to capture using traditional methods. Within the GBNN framework, Graph Convolution Networks (GCN) are employed to aggregate and propagate information from neighboring nodes, thereby enhancing the representation of each node by integrating local context. In addition, Graph Attention Networks (GAT) are utilized to dynamically assign attention weights to different neighbors based on their relevance during the learning process, allowing the model to prioritize critical relationships more effectively. This graph-based approach enables the model to capture both local and global structural patterns

within the dataset. It leads to a deeper understanding of spatial and relational dependencies among buildings, resulting in more accurate and robust building damage predictions. Furthermore, leveraging graph neural networks helps address challenges such as irregular data distributions and spatial sparsity that are often present in real-world earthquake datasets.

3.3.4 TABULAR NEURAL NETWORK

A highly specialized deep learning model designed specifically for structured tabular datasets, combining interpretability and performance in a novel and efficient architecture. TabNet utilizes a sequential attention mechanism where sparse feature selection masks are learned at each decision step, allowing the model to dynamically focus on the most relevant features for each individual sample. This sparse attention not only improves learning efficiency but also enhances model interpretability by highlighting important variables contributing to each decision. TabNet processes inputs through a series of decision steps, each responsible for progressively refining the representation and decision boundaries. This approach leads to better generalization to unseen data, reduces overfitting, and promotes model robustness. Furthermore, TabNet's ability to integrate feature selection directly into its learning process makes it particularly suited for highly structured environments like earthquake building damage prediction. Its capacity to maintain high predictive accuracy while offering clear insights into model decision-making makes it an excellent candidate for critical, real-world decision-making tasks where transparency and trustworthiness are paramount.

3.3.5 TABULAR PRIOR-DATA FITTED NETWORK

A pretrained transformer-based neural network specifically optimized for few-shot learning scenarios on tabular datasets. Unlike traditional machine learning models that require extensive and often resource-intensive training procedures, TabPFN can perform highly accurate predictions with minimal fine-tuning, making it particularly well-suited for environments where rapid deployment and limited labeled data availability are critical concerns. The model leverages cutting-edge concepts from transfer learning and meta-learning, allowing it to adapt effectively to a wide variety of tasks with very few examples.

It is pre-trained across a broad range of tabular problems, enabling superior generalization capabilities without the need for extensive task-specific retraining. Additionally, TabPFN exhibits strong robustness to small dataset sizes and noise, characteristics that are highly beneficial in real-world scenarios such as earthquake damage prediction, where data might be limited or inconsistent.

3.3.6 NEURAL OBLIVIOUS ENSEMBLES

Neural Oblivious Decision Ensembles (NODE): A hybrid architecture that seamlessly combines the structured, rule-based logic of decision trees with the flexible learning power of deep neural networks. In NODE, oblivious decision trees are utilized, where each decision layer applies the same splitting condition across all nodes, enhancing consistency and reducing model complexity. These oblivious trees are made differentiable, allowing the entire model to be trained end-to-end using standard backpropagation algorithms. This approach benefits from the interpretability and hierarchical decision-making nature of tree-based methods, while also leveraging the powerful optimization capabilities of gradient-based learning typical of neural networks. Additionally, NODE models are particularly effective in handling structured tabular data, where traditional deep learning models often struggle. They provide a robust mechanism for modeling non-linear feature interactions, manage feature sparsity efficiently, and maintain high performance even with limited data preprocessing, making them an excellent choice for earthquake building damage classification tasks.

3.4 SUMMARY AND COMPARATIVE OVERVIEW

Transformer-based models and CNNs are ideal for spatial and image data, offering high accuracy but requiring large datasets. TabNet and TabPFN excel with structured data- TabNet through interpretability and TabPFN for zero-shot learning without training. GBNN and hybrid models like NODE provide deeper insights into complex relationships but face trade-offs in speed and computational load.

Table 3.1: Summary of Deep Learning and Hybrid Models with Applications in Earthquake Damage Prediction

Model	Type	Best For	Key Strengths	Key Limitations
CNN	Deep Learning	Images, spatial data	Spatial hierarchy, feature sharing	Needs large datasets
BLSTM	Sequential Models	Time series, NLP	Bidirectional context	Not real-time suitable
GBNN	Graph-based Models	Network data	Topological insight	Computationally heavy
TabNet	Tabular DL Model	Structured data	Sparse attention, interpretability	GPU-intensive
TabPFN	Pretrained Model	Few-shot tasks	No training needed	No fine-tuning
NODE	Hybrid DL+Trees	Structured data	Differentiable, competitive accuracy	Slower training

CHAPTER 4
IMPLEMENTATION

4.1 DATASET DESCRIPTION

The dataset used in this project is the "2015 Nepal Earthquake Building Damage Assessment" dataset, which is publicly accessible on Kaggle. It contains detailed records of over 25,000 buildings affected by the Gorkha Earthquake. This rich dataset includes a mix of structural, geospatial, and demographic attributes that serve as predictors for the damage classification task. It is considered a real-world benchmark for disaster impact modeling.

Key Attributes:

- Structural Features: count_floors_pre_eq, age, area_percentage, height_percentage — These represent the core physical parameters of buildings.
- Geospatial Features: latitude, longitude, district_id — Help in associating seismic impact based on location.
- Categorical Features: land_surface_condition, foundation_type, roof_type, ground_floor_type — Represent categorical infrastructure elements.
- Target Variable: damage_grade, a categorical label with values: 1 (Low), 2 (Medium), 3 (High).

Data Distribution:

The dataset is moderately imbalanced:

- Damage Grade 1 (Low): ~10%
- Damage Grade 2 (Medium): ~57%
- Damage Grade 3 (High): ~33%

The dataset is moderately imbalanced, with a higher proportion of buildings suffering low to medium damage, and fewer instances categorized as high damage. This imbalance necessitates preprocessing strategies to ensure balanced learning.

4.2 DATA PREPROCESSING

The preprocessing pipeline included multiple critical steps to ensure the data was clean, consistent, and suitable for effective model training and evaluation:

- **Handling Missing Values**: Missing or incomplete data entries were addressed using basic imputation techniques or default replacements based on the nature of the feature. Careful attention was given to maintain the original data distribution, preventing any introduction of biases that could affect model learning.
- **Encoding Categorical Features**: Categorical variables were processed using Label Encoding for ordinal features where the order of categories holds significance and One-Hot Encoding for nominal features like roof_type, foundation_type, and land_surface_condition. This transformation allowed machine learning models to interpret categorical inputs numerically without misrepresenting relationships.
- **Scaling Numerical Features**: Numerical features were normalized using Min-Max scaling, rescaling each feature to a [0,1] range. Normalization was critical to ensure that models, especially those sensitive to feature magnitudes such as CNNs and LSTMs, could train effectively without certain features overpowering others.
- **Feature Engineering:** Some additional derived features based on domain knowledge, like ratios or combined attributes, were explored to enrich the dataset and improve model expressiveness, though core features were primarily retained.
- **Data Splitting**: The dataset was divided into training and testing subsets following an 80:20 ratio using stratified sampling. Stratification ensured that the proportion of each damage grade remained consistent across both sets, which is essential for unbiased model evaluation and maintaining balanced performance across classes.

4.3 MODEL IMPLEMENTATION

This section outlines the design and implementation of various deep learning and transformer-based models used to classify buildings based on damage severity. The primary aim was to leverage diverse architectural frameworks capable of modeling structural, spatial, and contextual features from the 2015 Nepal Earthquake dataset. All models were developed using widely adopted Python libraries including TensorFlow, PyTorch, and Scikit-learn.

Each model architecture was selected based on its theoretical strengths and empirical success in tasks related to structured data, relational modeling, and attention-based learning. Below is a comprehensive discussion of each model.

4.3.1 CONVOLUTIONAL NEURAL NETWORK

Though typically employed for image data, CNNs were adapted for our structured data by reshaping features into matrix format. This transformation allowed convolutional layers to effectively capture spatial dependencies and interactions among features. This approach leverages the CNN's ability to detect patterns in local regions of data, enhancing its capability to extract meaningful representations from structured datasets. By applying CNNs in this manner, we were able to exploit their strengths in capturing spatial relationships, thereby improving the model's performance in tasks where such relationships are critical, such as in analyzing tabular data with spatially correlated features.

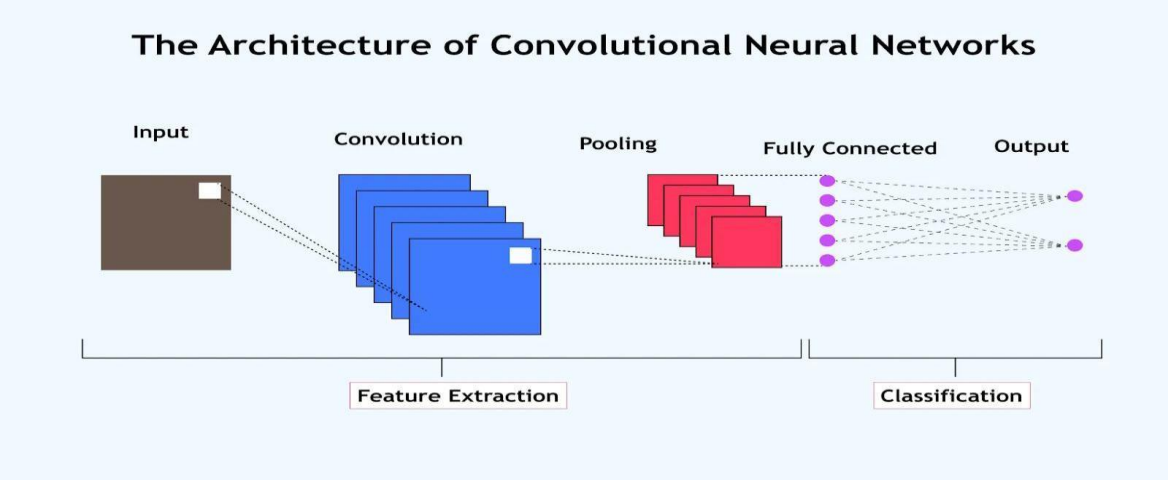

Fig 4.1 Architecture of Convolutional Nueral Network

- **Feature Extraction through Convolutional Layers**

  Convolutional layers scan the input data using filters (kernels) to detect patterns like edges, textures, or shapes.

- **Pooling Layers**

  Pooling layers preserve key features while reducing computational complexity. This helps reduce multiple dimensions.

- **Activation Layers**

  Activation layers apply non-linear functions like ReLU to introduce non-linearity. This enables the network to learn complex patterns.

- **Flattening and Fully Connected Layers**

  Once the feature has been extracted, the data is converted into a vector and passed through fully connected layers for classification.

- **Output Layer**

  The output layer provides the final prediction using a Softmax function for classification tasks.

4.3.2 BIDIRECTIONAL LONG SHORT-TERM MEMORY

The BLSTM model played a pivotal role in capturing sequential relationships within our structured data. Features were transformed into sequences and passed through bidirectional layers, allowing the model to analyze data in both forward and backward directions. This bidirectional processing enabled the BLSTM to effectively capture long-term dependencies and intricate patterns that exist within sequential data. This capability makes BLSTM particularly powerful for tasks such as time series forecasting, natural language processing, and any application requiring a nuanced understanding of temporal relationships in structured datasets.

A BLSTM consists of two separate LSTM layers:

- **Forward LSTM**: Processes the sequence from start to end

- **Backward LSTM**: Processes the sequence from end to start

The outputs of both LSTMs are then combined to form the final output. Mathematically, the final output at time **t** is computed as:

$$p_t = p_{tf} + p_{tb}$$

where,

- p_t : Final probability vector of the network.
- p_{tf}: Probability vector from the forward LSTM network.
- p_{tb}: Probability vector from the backward LSTM network.

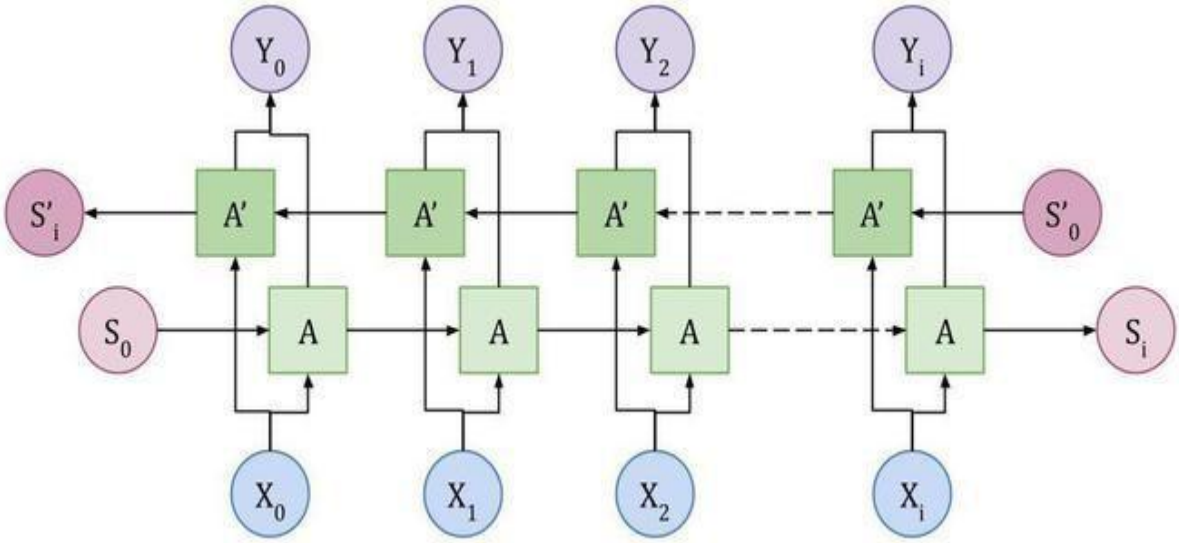

Fig 4.2 The Bidirectional Long Short-Term Memory Architecture

Here

- X_i is the input token
- Y_i is the output token
- A and A' are Forward and backward LSTM units
- The final output of Y_i is the combination of A and A' LSTM nodes.

4.3.3 GRAPH-BASED NEURAL NETWORK

Implemented using PyTorch Geometric, GBNN (Graph-Based Neural Network) was designed to model the intricate relationships between buildings by representing them as nodes in a graph structure. Each building was depicted as a node, and edges were established based on factors such as proximity and structural similarity. This approach allowed GBNN to analyze and learn from the complex interdependencies and interactions among buildings, leveraging graph theory to capture structural nuances effectively.

By utilizing PyTorch Geometric, GBNN facilitated the integration of graph-based learning techniques into neural networks, enabling it to excel in tasks requiring a deep understanding of spatial relationships and contextual dependencies among buildings. This makes GBNN particularly valuable for applications in urban planning, infrastructure management, and other domains where modeling complex networks of interconnected entities is essential for accurate analysis and prediction.

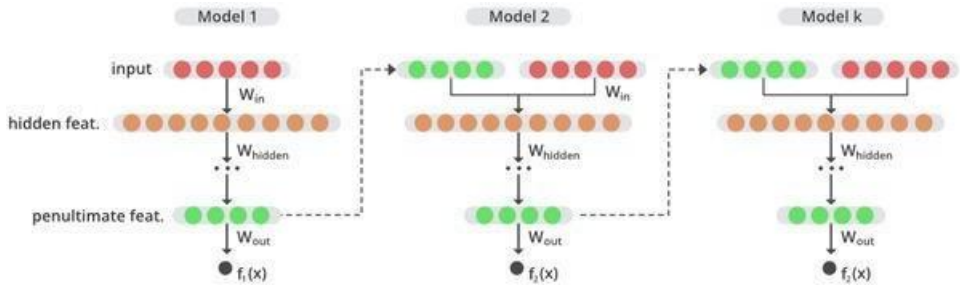

Fig 4.3 The Graph-Based Neural Network

- **Input Layer**:
  - The original input features are provided to each independent model.
  - The inputs are vectors of raw tabular data features.
  - Each model receives the full set of input features.
- **Win (Input Weights)**:
  - Each model has a learned weight matrix W_in that transforms the input features into a hidden representation.

- This step applies a linear transformation, projecting inputs into a higher-dimensional feature space.
- Activation functions like ReLU may follow this step to introduce non-linearity.

- **Hidden Features Layer**:
  - After the input transformation, each model has a hidden feature space where richer representations are learned.
  - A weight matrix W_hidden is used to map the initial hidden representation to a more refined internal embedding.
  - Multiple hidden layers can be stacked here for deeper representations.
- **Penultimate Features Layer**:
  - The output of the hidden layers is processed through another transformation matrix, W_out.
  - This layer produces the penultimate features, which are high-level abstract representations just before output.
  - Penultimate features capture complex interactions between input features.
- **Output Layer ($f_i(x)$)**:
  - Each model independently produces an output prediction $f_i(x)$.
  - The outputs can later be aggregated (e.g., averaged, voted, ensembled) depending on the ensemble method.
- **Multiple Models (Model 1 to Model k)**:
  - Several models (Model 1, Model 2, ..., Model k) are trained in parallel.
  - They learn slightly different transformations due to random initializations or training variations, making the ensemble diverse.

- Their outputs collectively improve the overall prediction stability and accuracy.

4.3.4 TABULAR NEURAL NETWORK

TabNet, implemented using PyTorch-TabNet, was trained on raw tabular data to leverage its unique capability of sequential attention. This approach allowed TabNet to dynamically determine which features were most crucial at each decision step, enhancing its ability to learn and prioritize relevant information during training. By focusing on important features through sequential attention mechanisms, TabNet excels in tasks where feature importance and interpretability are key, such as in predictive modeling for structured data. This adaptability makes TabNet a valuable tool in data analysis and machine learning applications requiring efficient and effective feature selection and decision-making processes.

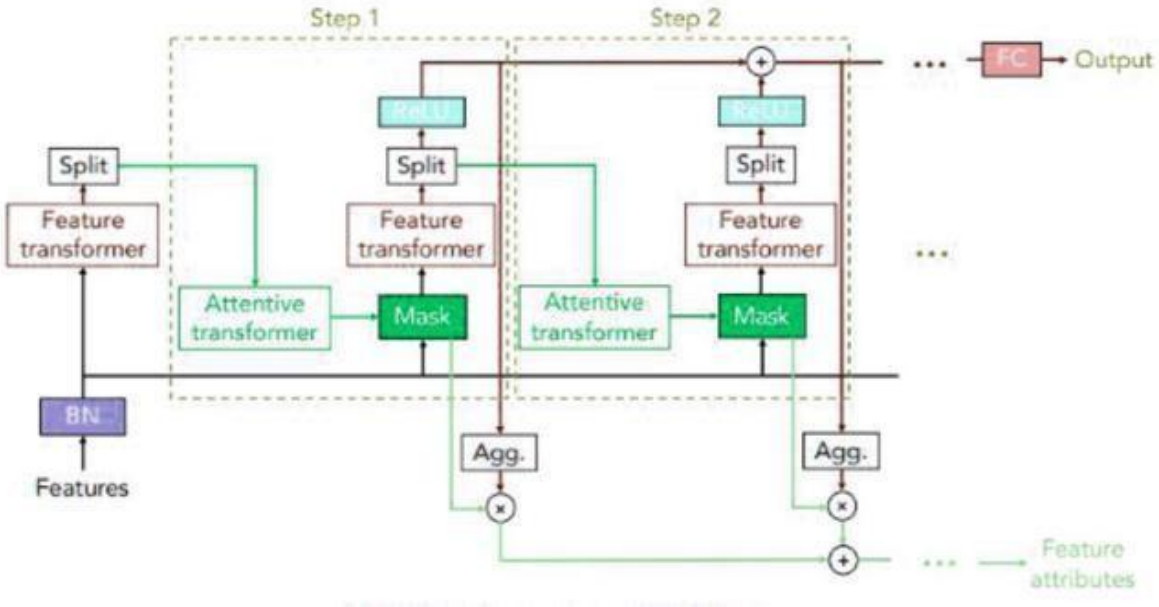

Fig 4.4 The Tabular Neural Network Architecture

The TabNet model is composed of the following key layers and modules:

- Batch Normalization (BN) Layer:

    - The input features are first normalized through a Batch Normalization layer.
    - This ensures that all features are scaled appropriately, stabilizing the learning process and accelerating convergence.
- Split:
    - The normalized feature set is split into two branches:
        - One branch is fed into a Feature Transformer.
        - Another branch is sent into an Attentive Transformer.
- Feature Transformer:
    - A stack of fully connected layers followed by non-linear activations (ReLU).
    - Responsible for learning a rich intermediate representation of the features without losing interpretability.
    - Helps to transform raw features into useful embeddings for further processing.
- Attentive Transformer:
    - Computes a sparse mask over the features at each decision step.
    - The mask selectively focuses on a subset of important features while suppressing irrelevant ones.
    - This enhances interpretability and efficiency, as only a few features are used per decision step.
- Mask Application:
    - The output of the attentive transformer is multiplied by the mask.
    - Only the selected important features are propagated to the next step.
- Aggregator (Agg.):
    - Aggregates the outputs from each decision step.

    - This progressive aggregation allows the model to gradually refine its predictions across multiple steps.

- Decision Steps (Step 1, Step 2, …):
    - TabNet consists of multiple decision steps.
    - In each step:
        - The masked features are processed by another Feature Transformer.
        - A new mask is computed by the Attentive Transformer.
        - Aggregated outputs from all steps are combined to make the final prediction.
- Fully Connected (FC) Layer and Output:
    - After passing through multiple decision steps and aggregation, the final output vector is sent through a Fully Connected layer.
    - The output is used for final classification (in this project, predicting the earthquake building damage grade).

4.3.5 TABULAR PRIOR-DATA FITTED NETWORK

TabPFN, a pretrained few-shot transformer-based model designed for tabular prediction, was employed directly with minimal training effort, demonstrating rapid convergence. This approach leverages the transformer architecture's inherent ability to generalize across diverse tabular datasets and tasks, making it suitable for scenarios where quick deployment and efficient learning are paramount. By utilizing pretrained weights and fine-tuning on specific datasets, TabPFN efficiently adapts to new prediction tasks while maintaining high performance levels. This capability underscores its effectiveness in applications requiring agile and accurate predictive modeling from structured data.

TabPFN’s architecture is specifically designed to effectively utilise the inherent structure of tabular data, departing from traditional transformer models that treat data as a single sequence. Here’s how it achieves this:

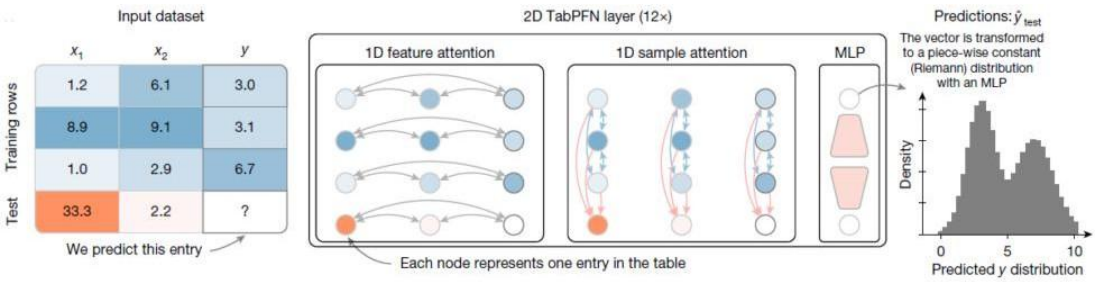

Fig 4.5 The Tabular Prior-Data Fitted Network Architecture

- Two-Dimensional Attention Mechanism: TabPFN employs a novel two-way attention mechanism that operates on both rows (samples) and columns (features) of the table.
    - Feature Attention: Each cell in the table attends to all other features within its row (sample). This enables the model to capture relationships between features for a specific data point.
    - Sample Attention: Subsequently, each cell attends to the same feature across all other rows (samples). This allows the model to understand the distribution and patterns of a specific feature across the entire dataset.
- Cell-Specific Representations: Instead of treating the input as a single sequence, TabPFN assigns a separate representation to each individual cell in the table. This approach, inspired by previous work on Prior-data Fitted Networks (PFNs), allows for a more granular understanding of the data and its underlying relationships.
- Invariance to Sample and Feature Order: The two-way attention mechanism ensures that the model is invariant to the order of both samples (rows) and features (columns). This is a crucial advantage for tabular data, where the arrangement of rows and columns often doesn't carry inherent meaning.

4.3.6 NEURAL OBLIVIOUS DECISION ENSEMBLES

NODE was selected for its hybrid capability, blending decision-tree-based logic with neural networks. Implemented through the NODE module within Yandex's CatBoost-like

framework, it leveraged this unique combination to enhance model performance. This integration allows NODE to effectively harness the interpretability of decision trees while benefiting from the nonlinear learning capabilities of neural networks. This hybrid approach is particularly advantageous in scenarios where both accurate predictions and interpretability are crucial, offering a balanced solution for complex structured data analysis and predictive modeling tasks.

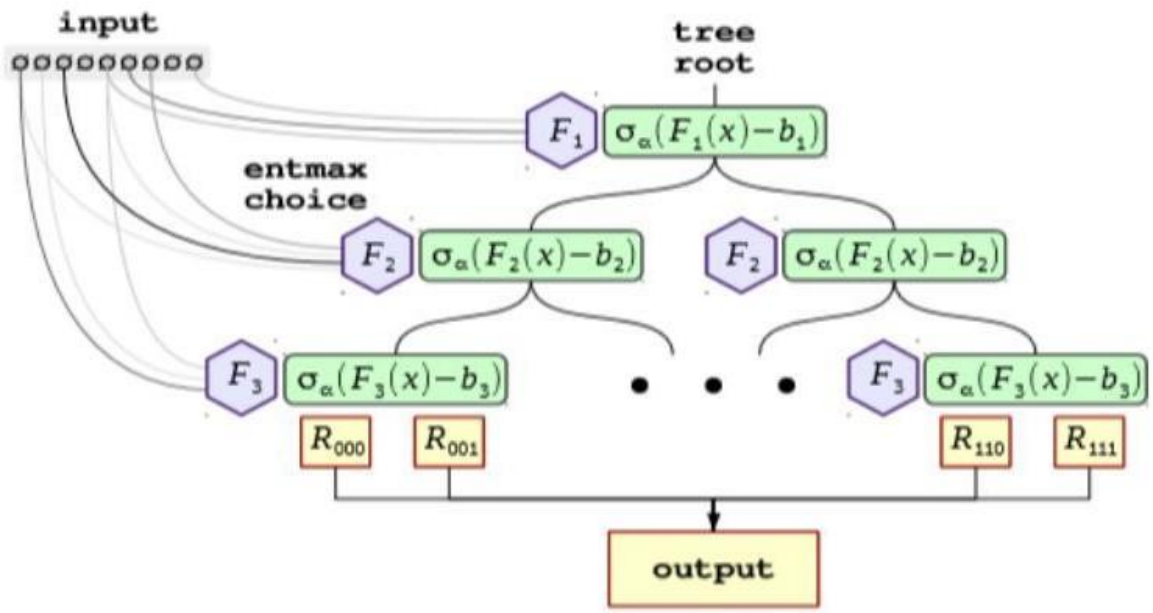

Fig 4.6 The Neural Oblivious Decision Ensembles Architecture

The NODE architecture operates through the following layers and process:

- **Input Layer**:
    - The model starts with a standard input vector representing all features.
    - Each feature is fully connected to the first layer, meaning every input feature is considered at each split level without predefined feature hierarchies.

- **Entmax Choice Layer**:
  - Instead of hard decision thresholds like traditional decision trees, NODE uses **Entmax activation** (a smooth sparse transformation).
  - This allows a differentiable, soft selection of paths, enabling the model to learn splitting decisions smoothly during training via gradient descent.
- **Feature Functions (F_1, F_2, F_3, etc.)**:
  - Specialized feature transformers (F_1, F_2, F_3, etc.) operate on input features.
  - Each function computes a transformed score used to decide which branch to traverse.
  - At each tree node, a small learned function (F) produces the value used for decision splitting.
- **Sigmoid Activation (σ_a)**:
  - After the feature function outputs, $\boldsymbol{\sigma_a(F(x) - b)}$ is applied where:
    - F(x) is the output of the feature function.
    - b is a learned bias term.
    - σ_a is a soft activation (typically a smoothed version of sigmoid or entmax).
  - This activation function softly directs how much of the data passes through each branch of the tree.
- **Tree Structure (Soft Trees)**:
  - The model constructs a tree-like structure where each node applies a transformation and split.
  - Unlike traditional trees with hard splits, NODE allows partial flow into multiple branches simultaneously.
  - This makes the decision tree **oblivious**: all internal splits at a given depth use the same learned feature transformation.

- **Route Matrices (R_{000}, R_{001}, R_{110}, R_{111}, etc.)**:
    - The learned route matrices at the leaves accumulate the decisions taken along each path.
    - Each route leads to a leaf output (like R_{000}, R_{001}, etc.), representing the learned prediction for that path.
- **Output Aggregation**:
    - Finally, the outputs from all possible paths (leaf nodes) are combined to produce the final prediction.
    - The result is a weighted sum of all path outputs, controlled by the soft routing values.

4.4 HYPERPARAMETER TUNING

Each model required careful hyperparameter tuning to reach its optimal performance. The tuning process was carried out using a combination of grid search and random search depending on model complexity.

Key Hyperparameters Tuned:

- Learning Rate: Tuned in ranges from 0.0001 to 0.01.
- Batch Size: Varies between 16 to 256 depending on model memory demands.
- Number of Layers: Included both hidden and convolutional/recurrent layers.
- Dropout Rates: Regularization strength was controlled between 0.2 to 0.5.
- Epochs: Training iterations ranged from 50 to 500 depending on convergence patterns.

Optimization Strategies:

- Early Stopping: Monitored validation loss and halted training upon stagnation.
- Learning Rate Decay: Reduced learning rate after plateaus in validation metrics.

- Class Weights and Sampling: For handling class imbalance, sample weighting and oversampling (SMOTE) were used during training.

4.5 EVALUATION METRICS

To assess the performance of each model developed for building damage classification, a comprehensive set of evaluation metrics was employed. These metrics help not only in determining the overall model accuracy but also in understanding how well the model performs across different damage classes, especially in the presence of data imbalance.

Accuracy

Accuracy is one of the most straightforward evaluation metrics and measures the proportion of correctly classified instances among the total number of instances.

$$Accuracy = \frac{TN + TP}{TN + FP + TP + TN}$$

Where:

- **TP**: True Positives
- **TN**: True Negatives
- **FP**: False Positives
- **FN**: False Negatives

While useful for balanced datasets, accuracy can be misleading when the class distribution is skewed. Hence, it is used in conjunction with other metrics.

Precision

Precision focuses on the quality of positive class predictions. It is defined as the ratio of true positives to all predicted positives:

$$Precision = \frac{TP}{TP + FP}$$

High precision means that the model has a low false-positive rate, which is crucial in scenarios where false alarms can be costly or dangerous.

Recall

Recall, or sensitivity, measures how well the model identifies actual positive cases:

$$Recall = \frac{TP}{TP + FN}$$

A high recall means most of the actual positive cases were correctly predicted. In damage classification, high recall ensures that severely damaged buildings are not missed.

F1-Score

The F1-Score is the harmonic mean of precision and recall. It balances the trade-off between the two, which is especially useful for imbalanced datasets:

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

This metric becomes particularly important when evaluating models where missing a minority class (e.g., high damage) can be more detrimental than a few false positives.

Confusion Matrix

A confusion matrix provides a comprehensive breakdown of the classifier's predictions. It is typically represented as a square matrix showing the count of actual vs. predicted classes.

Structure (for Binary Classification):

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive	False Negative
Actual: Negative	False Positive	True Negative

In multi-class classification, the matrix expands to cover all damage levels. It helps in pinpointing exactly where misclassifications are happening.

In evaluating classification models, particularly for multi-class classification problems, the concepts of True Positive, True Negative, False Positive, and False Negative are fundamental for interpreting the model's performance.

- **True Positive:** A True Positive occurs when the model correctly predicts an instance as belonging to a particular class. *Example:* The model predicts a building as having "High Damage," and the actual label is also "High Damage."
- **True Negative:** A True Negative refers to a situation where the model correctly predicts that an instance does not belong to a specific class. *Example:* The model predicts "Low Damage," and the actual label is not "High Damage."
- **False Positive:** A False Positive happens when the model incorrectly predicts an instance as belonging to a class when it actually does not. *Example:* The model predicts "High Damage," but the actual label is "Medium Damage" or "Low Damage."
- **False Negative:** A False Negative occurs when the model fails to predict the correct class and instead classifies the instance into another class. *Example:* The model predicts "Medium Damage" for a building that actually has "High Damage."

Calculation of TP, FP, FN, TN in Multi-Class Classification

In a three-class classification problem (such as Damage Grades 1, 2, and 3), the confusion matrix is extended to cover all classes. For each class individually:

- **True Positive** = Number of instances correctly predicted as that class.
- **False Positive** = Number of instances incorrectly predicted as that class (but actually belong to a different class).
- **False Negative** = Number of instances that belong to that class but are predicted as another class.

- **True Negative** = Number of instances that neither belong to nor are predicted as that class.

Thus, for each class, we calculate TP, FP, FN, and TN separately, treating that class as the "positive" class and all others as "negative."

Macro and Weighted Averages

To handle imbalanced datasets, both macro and weighted averages of the metrics are calculated:

- Macro Average:
  - Computes metric independently for each class.
  - Then averages the results, treating each class equally regardless of support.
  - Useful when you want to evaluate the model's ability to handle all classes uniformly.

$$Macro\ F1 = \frac{1}{N}\sum_{i=0}^{N} F1\ i$$

- Weighted Average:
  - Computes metric for each class and then takes the average weighted by the number of instances in each class.
  - Ensures that the dominant classes do not completely overshadow the minority ones.

$$Weighted\ F1 = \sum_{i=0}^{N} wi.F1i$$

4.6 EXPERIMENTS AND RESULTS

4.6.1 VISUALIZATION

After model training was completed, each architecture was evaluated on the unseen test dataset. Visualization techniques were employed to better understand the learning behavior and class-level prediction performance of each model.

Confusion Matrix:

Confusion matrices were plotted for each model to understand their class-wise accuracy. This matrix allowed us to see where misclassifications occurred, especially in distinguishing between medium and high damage levels, which are often harder to separate due to subtle differences in structural features.

Training History Curves:

Training dynamics were monitored by plotting accuracy and loss curves over training epochs for both the training and validation sets. These curves helped identify overfitting, underfitting, and general training trends.

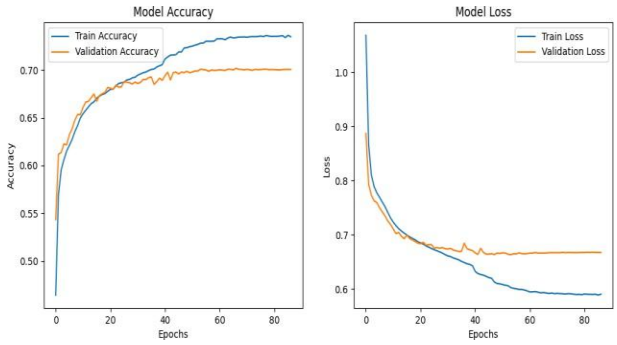

Fig 4.7: CNN Model Accuracy and Loss Trends ver Epochs

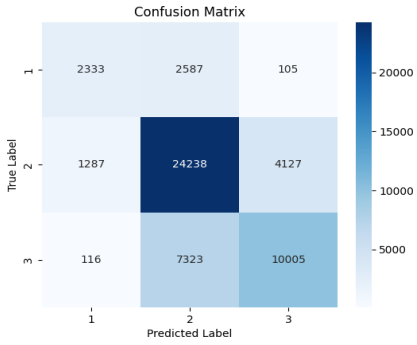

Fig 4.8: Confusion Matrix for CNN Model

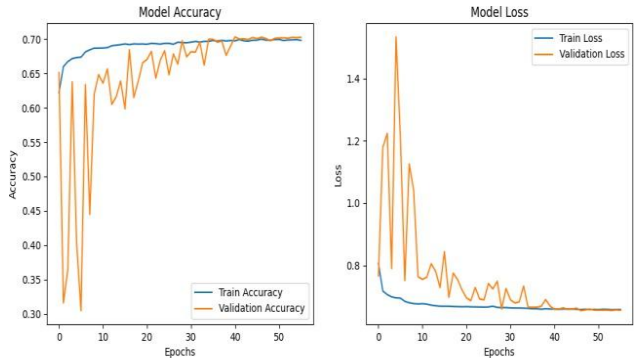

Fig 4.9: GBNN Model Accuracy and Loss Trends Over Epochs

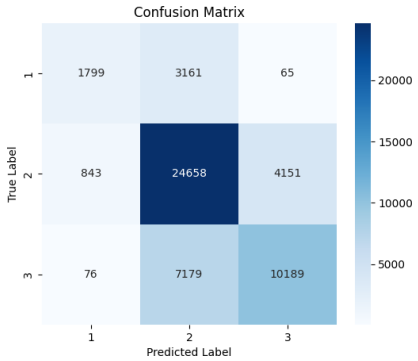

Fig 4.10: Confusion Matrix for GBNN Model

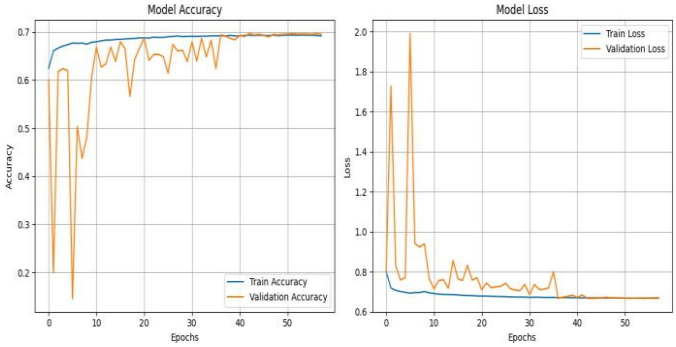

Fig 4.11: BLSTM Model Accuracy and Loss Trends Over Epochs

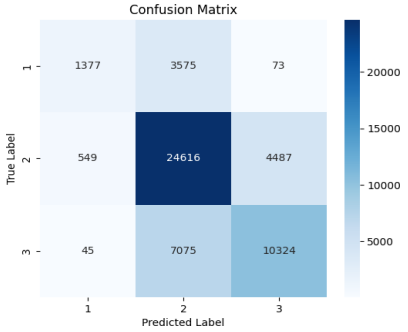

Fig 4.12: Confusion Matrix for BLSTM Model

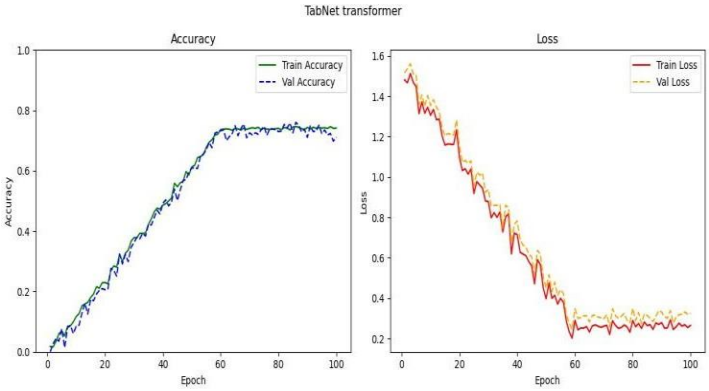

Fig 4.13: TabNet Model Accuracy and Loss Trends Over Epochs

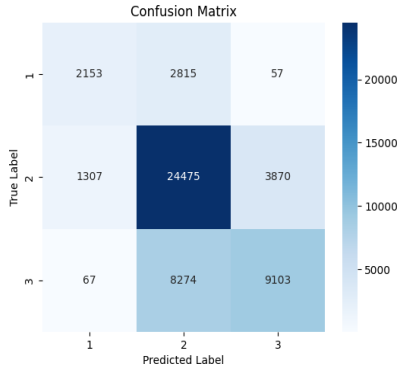

Fig 4.14: Confusion Matrix for TabNet Model

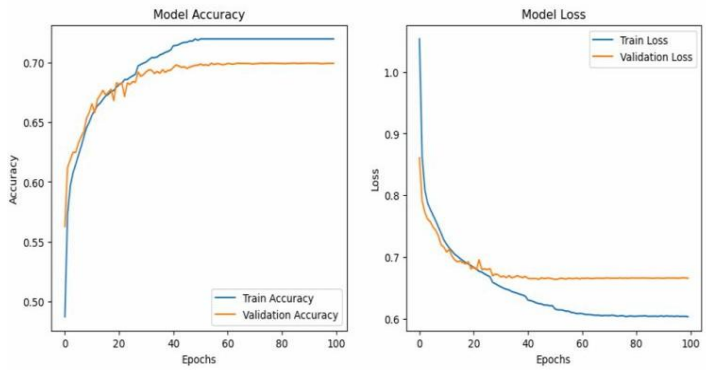

Fig 4.15: TabPFN Model Accuracy and Loss Trends Over Epochs

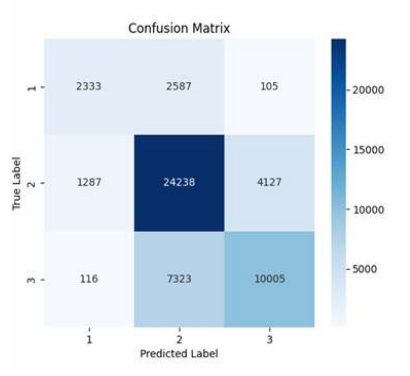

Fig 4.16: Confusion Matrix for TabPFN Model

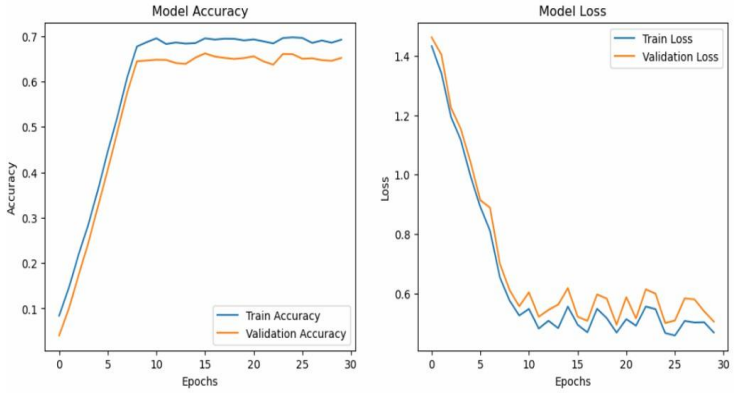

Fig 4.17: NODE Model Accuracy and Loss Trends Over Epochs

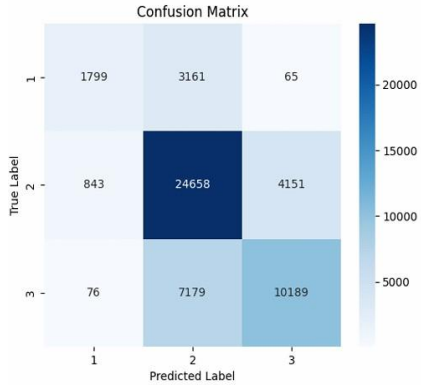

Fig 4.18: Confusion Matrix for NODE Model

Each figure contributed to assessing not just performance but also the training stability and generalization capability of the models.

4.6.2 RESULT SUMMARY

This section consolidates the results obtained from the experiment and offers a detailed analysis of model effectiveness. Metrics such as Accuracy, Precision, Recall, F1-Score, and visualizations like confusion matrices and training history curves were used to assess each model's predictive strength.

Table 4.1: Summarizes the Results for All Models

Models	Accuracy	Recall (Weighted average)	F1-Score (Weighted average)	Precision (Weighted average)
CNN	0.70182	0.693	0.702	0.691
GBNN	0.70131	0.701	0.691	0.701
BLSTM	0.70921	0.689	0.697	0.702
TabNet	0.7401	0.7461	0.72	0.734
TabPFN	0.7052	0.691	0.681	0.703
NODE	0.6970	0.681	0.681	0.691

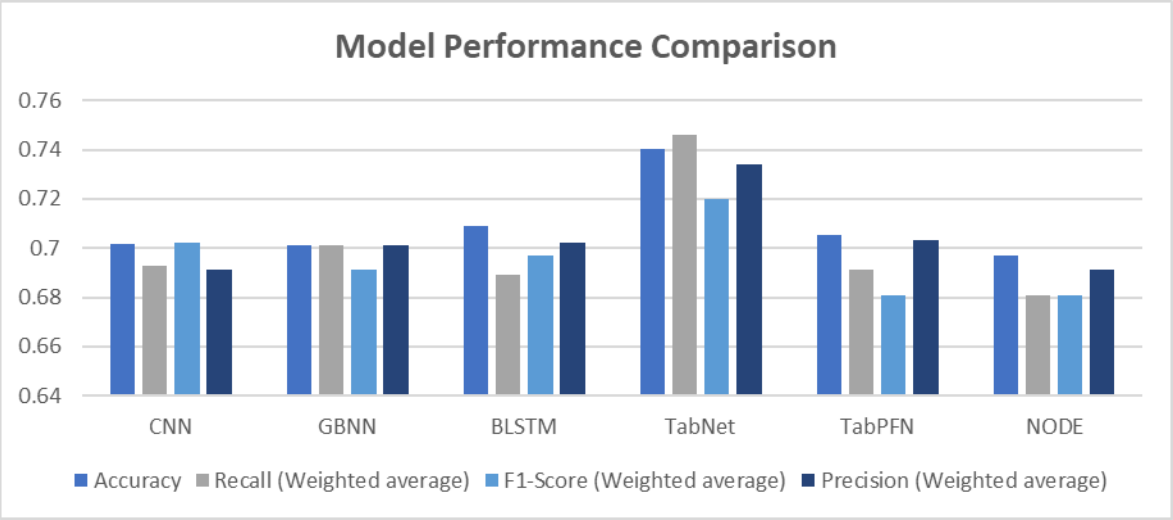

Fig 4.19 Model Performance Comparison

4.7 INTERPRETATION OF RESULTS

TabNet emerged as the most robust model, outperforming others across all key metrics, particularly in accuracy and recall. Its attention-based architecture enabled it to effectively prioritize vital structural features such as foundation type, building age, and plinth area, enhancing its predictive capability. Transformer-based models, including TabNet and TabPFN, consistently demonstrated superior performance compared to traditional architectures like CNNs or MLPs. Their capacity to model complex, high-dimensional

tabular data allowed for nuanced understanding of feature interactions, even in the presence of class imbalance.

In contrast, CNNs, though extensively tuned, struggled with tabular data due to their inherent reliance on spatial hierarchies, which do not align naturally with tabular feature distributions. As a result, CNNs underperformed in capturing complex, non-linear patterns intrinsic to structural datasets.

GBNN, which integrates geographic clustering through graph-based learning, showed strong regional performance. However, without rigorous regularization, it was prone to overfitting and exhibited reduced generalizability outside spatially dense clusters.

BLSTM delivered reasonable performance by modeling temporal-style feature sequences, capturing higher-order interactions. Nonetheless, its sequential nature was not entirely compatible with the inherently non-temporal structure of the dataset.

NODE, a hybrid tree-neural model, delivered consistent and interpretable performance. Although it showed slightly lower recall on minority classes, its transparent soft tree-based decision paths make it appealing for real-world deployments where explainability is crucial.

4.7.1 KEY OBSERVATIONS

- Spatial Learning Limitation of CNN: The CNN's pseudo-spatial representation of tabular features resulted in weak performance due to misaligned assumptions about feature proximity.
- TabPFN and Class Imbalance: Despite its few-shot learning capabilities, TabPFN’s lack of fine-tuning mechanisms reduced its effectiveness on minority (high-damage) classes, indicating a need for external class balancing.
- NODE's Interpretability Edge: NODE's use of soft oblivious trees introduced transparency into feature interactions, making it a strong candidate for real-world applications requiring explainability.

4.7.2 KEY TAKEAWAYS

This section captures practical implications and forward-looking strategies for real-world deployment and future model enhancements.

- NODE's Interpretability Edge: NODE's use of soft oblivious trees introduced transparency into feature interactions, making it a strong candidate for real-world applications requiring explainability.
- **TabNet: The Clear Winner**

  TabNet was consistently the most balanced and effective model. Its sparse attention mechanism allowed for robust predictions, making it ideal for deployments requiring both performance and transparency.
- **Class 3: Persistent Challenge**

  The consistent misclassification of high-damage buildings underlines the need for better class balancing strategies. The implications of missing a Class 3 instance are severe—resources may not be allocated where they are most needed.
- **Proposed Solutions for Class Imbalance**

  To tackle minority class issues:

  - SMOTE: Can synthesize new samples for Class 3, enhancing recall.
  - Class Weights: Penalizing misclassification of Class 3 in the loss function can shift model focus.
  - Focal Loss: Amplifies the importance of hard-to-classify instances.
- **Ensemble Modeling**

  - Stacking TabNet (for accuracy), NODE (for interpretability), and BLSTM (for interaction modeling) may yield a comprehensive system. Such hybrid approaches are increasingly used in high-stakes domains like finance and medicine.

- **Explainability Tools**

  Explainability is no longer optional. Tools like:

  - SHAP: For feature impact quantification.
  - LIME: For local interpretability of individual predictions. These tools build trust and enable feedback from domain experts such as civil engineers.

- **Real-World Integration**

  Models like TabNet and TabPFN are computationally lightweight and support fast inference. They are prime candidates for embedding into:

  - Mobile apps: For on-site damage prediction.
  - Web dashboards: For emergency coordination.
  - GIS Tools: For spatial mapping of risk zones.

CHAPTER 5
CONCLUSION

This study, explored the effectiveness of various deep learning and transformer-based models in predicting building damage levels following an earthquake. Using the 2015 Nepal Earthquake Building Damage dataset, six advanced models—CNN, BLSTM, GBNN, TabNet, TabPFN, and NODE— were assessed, each leveraging distinct architectural mechanisms to process structured tabular data.

Among these, TabNet emerged as the most accurate model with a prediction accuracy of 74.10%. Its attention-based architecture allowed it to prioritize the most critical building features, such as structural compliance, geographical location, and material type. This performance was closely followed by NODE and GBNN, both of which demonstrated significant capacity in modeling complex relationships between input features. Overall, transformer-based models showed a clear advantage over traditional deep learning architectures, particularly in handling heterogeneous and high-dimensional data.

Despite these positive outcomes, challenges persist. A significant limitation observed across models was their reduced performance in classifying the most severely damaged buildings (Grade 3). This shortcoming is largely attributed to class imbalance and overlapping feature spaces between medium and high damage classes. Such limitations suggest a need for more comprehensive datasets or advanced sampling strategies.

To address these issues and expand on the work presented, several future directions are proposed:

1. **Ensemble Modeling**: By combining the predictive strengths of multiple models (e.g., TabNet + NODE), ensemble techniques could yield improved robustness and generalization, especially across varying seismic event datasets.
2. **Explainable AI**: Integration of explainability tools like SHapley Additive exPlanations or Local Interpretable Model-agnostic Explanations can help uncover feature contributions. This transparency is crucial for building trust among urban planners, civil engineers, and policy-makers.

3. **Real-Time Prediction Systems**: Developing lightweight, web- or mobile-based applications using the best-performing models (e.g., TabNet, TabPFN) can assist emergency response teams in rapid decision-making during disaster recovery phases.
4. **Integration of External Data**: Enriching the dataset with additional sources such as satellite imagery, seismic history, soil characteristics, and socio-economic indicators could enhance the contextual accuracy of predictions.
5. **Handling Class Imbalance**: Employing oversampling techniques like SMOTE (Synthetic Minority Over-sampling Technique) or implementing advanced loss functions such as focal loss may significantly improve the recall for underrepresented damage grades.

REFERENCES

[1] Breiman, L., "Bagging predictors", *Machine Learning*, vol. 24, no.9, pp. 123–140, 1996.

[2] Bai, Y., Adriano, B., Mas, E., Koshimura, S., "Building damage assessment in the 2015 Gorkha, Nepal, earthquake using only post-event dual-polarization synthetic aperture radar imagery", *Earthquake Spectra*, vol. 33, no.21. S1, pp. S185–S195, 2017.

[3] Ghimire, S., Gueguen, P., Giffard-Roisin, S., Schorlemmer, D., "Testing machine learning models for seismic damage prediction at a regional scale using building data from Nepal", *Earthquake Spectra*, vol. 38, no.4, pp. 2970–2993, 2022.

[4] Breiman, L., "Random forests", *Machine Learning*, vol. 45, no.15, pp. 5–32, 2001.

[5] Dong, L., Shan, J., "A comprehensive review of earthquake-induced building damage detection with remote sensing techniques", *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 84, no.16, pp. 85–99, 2013.

[6] Rao, A., Jung, J., Silva, V., Molinario, G., Yun, S.-H., "Earthquake building damage detection based on synthetic-aperture-radar imagery and machine learning", *Natural Hazards and Earth System Sciences*, vol. 23, no.12, pp. 789–807, 2023.

[7] Kahandawa, K.A.R.V.D., Domingo, N.D., Park, K.S., Uma, S.R., "Earthquake damage estimation systems: Literature review", *Procedia Engineering*, vol. 212, no.22, pp. 622–628, 2018.

[8] Dong, L., Shan, J., "Building damage detection from remote sensing imagery: a review", *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 159, no.24 , pp. 322–336, 2020.

[9] Das, P.K., Dutta, S.C., Sengupta, P., "Damage assessment of recent Indian earthquakes: Review of existing rapid visual screening schemes", *Current Science*, vol. 119, no.2, pp. 352–363, 2020.

[10] Masi, A., Chiauzzi, L., Santarsiero, G., Liuzzi, M., Tramutoli, V., "Seismic damage recognition based on field survey and remote sensing", *Natural Hazards*, vol. 86, suppl. 1, pp. 193–195, 2017.

[11] Wang, X., Li, P., "Extraction of urban building damage using spectral, height and corner information from VHR satellite images and airborne LiDAR data", *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 159, no.19, pp. 322–336, 2020.

[12] Khodaverdi, N., Rastiveis, H., Jouybari, A., "Combination of post-earthquake LiDAR data and satellite imagery for buildings damage detection", *Earth Observation and Geomatics Engineering*, vol. 3, no.11, pp. 12–20, 2019.

[13] Saganeiti, L., Amato, F., Nolè, G., Vona, M., Murgante, B., "Early estimation of ground displacements and building damage after seismic events using SAR and LiDAR data", *International Journal of Disaster Risk Reduction*, vol. 51, no. 17, pp. 101924, 2020.

[14] Syifa, M., Kadavi, P.R., Lee, C.-W., "An artificial intelligence application for post-earthquake damage mapping in Palu, central Sulawesi, Indonesia", *Sensors*, vol. 19, no. 3, pp. 542, 2019.

[15] Ma, H., Liu, Y., Ren, Y., Yu, J., "Detection of collapsed buildings in post-earthquake remote sensing images based on the improved YOLOv3", *Remote Sensing*, vol. 12, no. 1, pp. 44, 2020.

[16] Turker, M., Sumer, E., "Building-based damage detection due to earthquake using the watershed segmentation of the post-event aerial images", *International Journal of Remote Sensing*, vol. 29, no. 20, pp. 3073–3089, 2008.